

Capstone Project Milestone 1: Discovery Report

DataSage: An Agentic System for Data Quality, Cleaning, and Analysis · Group 25

Team repository: <https://github.com/shrey-Bish/cse598-group25-datasage>

[DRAFT: replace every highlighted item after the stakeholder meeting, then delete this line.]

Basic Information

Student name

Shrey Bishnoi

[Student 2 name]

[Student 3 name]

[Student 4 name, if any]

[Student 5 name, if any]

Stakeholder name

Vyshanth Buddani

Email

[ASU email]

[email]

[email]

[email]

[email]

Meeting Date

[meeting date]

Section 1. Refined Problem Definition and Scope

Problem. Exported spreadsheets (sales exports, survey results, course records) often have missing values, repeated rows, labels typed in different ways, broken dates and numbers that do not add up. Analyzed as they are, they give wrong answers and nothing warns the user. The stakeholder's baseline shows this on its own example file. It detects every planted problem, but it still reports Accessories revenue as \$1,350, because a duplicate row is counted twice and the lowercase "accessories" row is kept separate. After removing the duplicate and merging the labels, the total is \$830. Electronics is \$2,185 if a negative webcam row is a refund, or \$2,245 if that row is an error and is excluded; only the user can say which. Cleaning by hand fixes this, but it is slow and the decisions are rarely written down.

DataSage will take a messy table and a question, find the problems that matter for that question, fix the safe ones, ask the user about risky ones, check that each fix did what it should, and answer the question on the cleaned data with a record of every change.

Intended users. Analysts, students and small-business staff who are comfortable with spreadsheets but do not want to write cleaning code. They need a correct answer, and they need to be able to explain where it came from.

Use scenario. A user uploads the August sales export and asks "What was total revenue by category?" DataSage reports one duplicate row, two labels that differ only by capitalization, one date it cannot read, one row with negative revenue, and one missing region, which does not affect this question. It merges the labels on its own, asks before removing the duplicate and before excluding the negative row, then answers the question and lists every change it made.

Input.

- One table as CSV or XLSX (one sheet, one header row), up to about 100,000 rows and 50 columns.
- An analysis request in plain English.
- Optional: a short rules file describing columns and valid values, for example "Revenue = Quantity × Price" or "Region is one of North, South, East, West".

Output.

- A quality report: each problem found, with its type, column and rows, and what was done about it (fixed, flagged or left alone).

- A change log: every change with its reason, the rows it touched, and whether the user approved it.
- A cleaned copy of the file. The original file is never modified.
- The answer to the request as a table, with a short summary and any caveats.
- A short pandas script that replays the cleaning, so the result can be reproduced without the agent.

Tasks. Profile the table, detect problems, decide for each one whether to fix, flag or ignore it, apply and check fixes on a working copy, run the requested analysis, and write the report (Section 3.2).

Scope. In scope: single-table CSV and Excel files; missing values, exact and near-duplicate rows, label variants (case, spaces, spelling), invalid or mixed date formats, numbers stored as text, negative or out-of-range values, and cross-column rule violations; and any analysis pandas can express (filters, group-by summaries, time summaries, basic statistics). Out of scope: unstructured data, joining several tables, data too large for memory, machine-learning models, and writing changes back to the source system. JSON input is a stretch goal.

Success and failure. From the stakeholder's point of view, success means the answer matches the answer on correctly cleaned data, every change in the cleaned file can be traced to a logged reason, nothing destructive happens without approval, and the user is not asked about obvious fixes. Failure means a wrong number shown with confidence, a silent change to the data, a made-up column or value, a fix that changes what the data means (for example dropping valid refunds), or the system giving up or looping without an answer. A silent wrong change is worse than an unanswered question, so when DataSage is unsure it should flag rather than fix.

Section 2. Stakeholder Discovery and Feedback

Meeting. [Date, time, length and format (in person or Zoom). Attendees: Vyshanth Buddani and the team members present.]

Stakeholder's needs, current workflow, constraints and priorities. [Fill in from the meeting: who he imagines using DataSage, how that person cleans data today, limits such as time, cost or data privacy, and what matters most to him.]

Before the meeting we studied the proposal and ran the baseline ourselves (Section 5). From that we prepared six refinements to confirm with the stakeholder. Table 1 lists them; the last column records his decision.

Table 1. Refinements proposed to the stakeholder.

#	Proposed refinement	Why	Stakeholder decision
1	Clean a copy, never the original, and log every change with its reason.	The proposal's success criterion is "no unsupported data changes". A change log lets us check that exactly.	[confirmed / changed / rejected]
2	Apply safe formatting fixes automatically; ask before risky ones (removing rows, filling values, changing numbers, fuzzy label merges). See Table 2.	The proposal plans "user approval for ambiguous changes". We need to agree where the line is.	[decision]
3	Replace open retrieval of external definitions with an optional rules file supplied by the user.	Rules found online may not fit the user's data and are hard to check. A rules file is explicit and testable.	[decision]
4	Support any analysis that can be written as filter, group by and aggregate, with sandboxed pandas code as a fallback.	Replaces the single fixed revenue analysis with general requests, while keeping results checkable.	[decision]

#	Proposed refinement	Why	Stakeholder decision
5	Evaluate mainly on public tables with planted errors, plus a few real messy files from the stakeholder.	Real files rarely come with correct answers. Planted errors give exact ground truth.	[decision]
6	Use the metrics and targets in Section 6 as the definition of success.	Success metrics must be agreed with the stakeholder.	[decision]

Main feedback and how it changed the proposal. [Summarize what he confirmed, what he changed, and anything new he asked for. If he rejected a refinement in Table 1, update the matching section.]

Stakeholder confirmation. [For example: "Vyshanth confirmed the refined scope and success metrics by message on <date>."] Meeting notes and the agenda we used are in Appendix A.

Section 3. Agentic System Design and Technical Approach

3.1 Why an agent, not one model call or a fixed workflow

We considered two simpler designs.

- **One model call.** The model cannot see a whole file (100,000 rows do not fit in its context, and sending them raises privacy concerns), cannot add up thousands of numbers reliably, and cannot check whether its own fixes worked.
- **A fixed workflow.** This is what the baseline is. It runs the same checks on every file, so it misses anything outside its rules. We wrote a 5-row probe file with four planted problems (eval/probes/hidden_issues.csv in our repository). The baseline caught only one. It missed a category with a trailing space, an invalid date in a column called "Day" (it only checks columns whose names contain "date" or "time"), and a revenue value that did not equal quantity times price.

The right action depends on the file and on the question. A missing Region does not matter for revenue by category but does for revenue by region. A negative quantity may be a refund to keep or a typo to fix. These choices need judgment, and they need feedback: after each fix, the system has to look at the result and decide what to do next. That observe-and-decide loop is why we need an agent. Our design rule is to keep judgment in the model and all computation in ordinary code.

3.2 Workflow

Figure 1 shows the full workflow. One run goes like this:

1. **Load and profile.** pandas loads the file and a profiler computes, for every column, its type, missing rate, distinct values, how many values parse as dates or numbers, and label variants. The model sees this profile and up to 20 sample rows chosen to include unusual values, never the full table.
2. **Plan and check.** The planner agent reads the profile, the request and the rules file, then calls check tools on the columns it thinks matter.
3. **Propose fixes.** For each problem worth fixing, the planner proposes one fix from a fixed set of fix tools, with a reason.
4. **Approval gate.** Code decides the risk of each fix from its type, so the model cannot argue its way past it (Table 2). Safe fixes run directly; risky fixes are shown to the user. A rejected fix becomes a flag in the report.
5. **Apply.** Each fix runs on a working copy and creates a new version (v1, v2, ...) plus a change-log entry.
6. **Validate.** The validator re-runs the related check and tests invariants: rows removed match what the fix said, other columns are unchanged, and totals reconcile (before minus removed

equals after). On failure the version is rolled back and the failure goes back to the planner as an observation. Steps 2 to 6 repeat.

7. **Analyze.** The planner answers the request with the analyze tool, which takes structured arguments (group by, metric, aggregate, filter). For anything else it can write pandas code that runs in a sandbox: a separate process with a time limit, no network and read-only data.
8. **Report.** The report builder writes the outputs from Section 1. A final check compares every number in the summary with the tool results, so no number can appear that a tool did not produce.

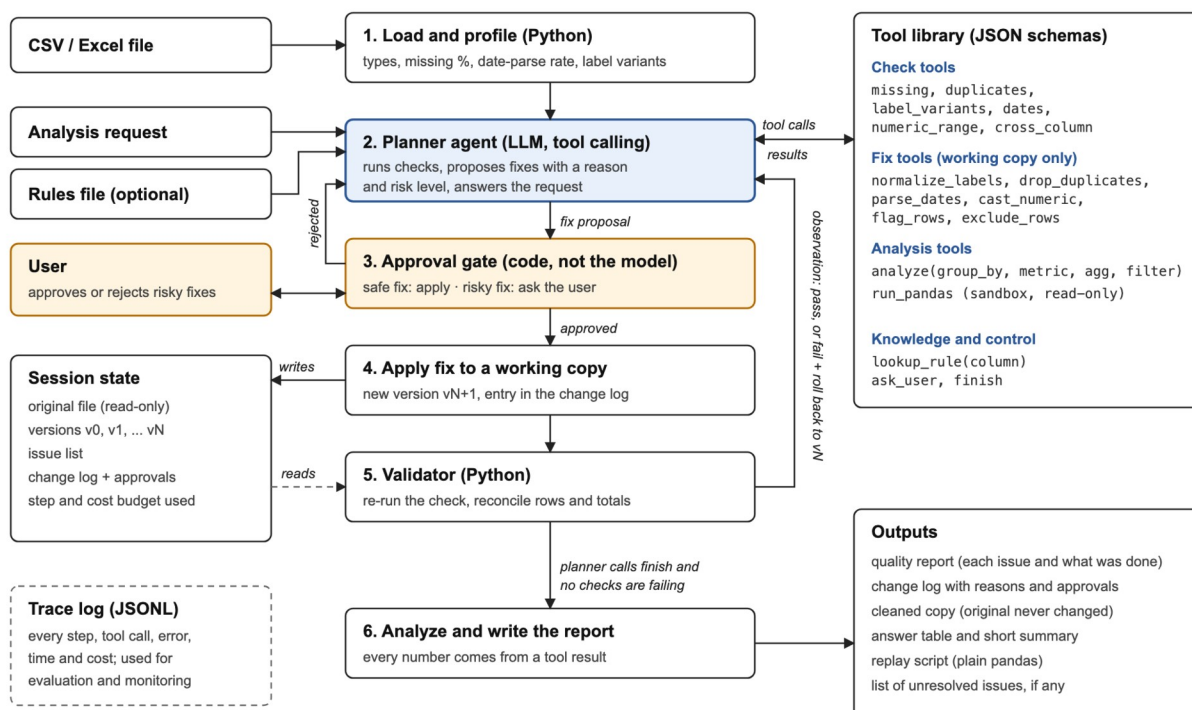


Figure 1. DataSage architecture. Blue is the only component that uses a language model; orange is where a person or the risk policy decides.

Table 2. Proposed risk tiers for fixes (to be confirmed by the stakeholder, Table 1 item 2).

Fix	Risk	Handling
Trim spaces; merge labels that are identical after lowercasing and trimming	Low	Applied and logged
Parse dates in an unambiguous format; convert numbers stored as text (for example "\$1,200")	Low	Applied and logged
Remove exact duplicate rows	Medium	Ask the user
Exclude or flag rows (negative values, out of range, rule violations)	High	Ask the user
Fill missing values; change a number; merge labels by fuzzy match ("Elec." to "Electronics")	High	Ask the user

3.3 Tools and function calling

Every tool (listed in Figure 1) has a JSON schema with a name, description and parameters, which is sent to the model, and a Python function that does the work. Arguments are validated with pydantic before the function runs. A failed call returns the error to the model as text, for example "column 'Revenue' not found; columns are Order_ID, Date, ...", so it can correct itself instead of crashing the run. This is the pattern we saw when reading the Codex source in Lab 5, where tool

errors go back to the model as ordinary results. Results are kept short (counts and at most five example rows); for example, `normalize_labels(column, mapping)` returns the rows changed and the value counts before and after.

3.4 State, actions, observations and stopping

- **State:** the profile, open issues, the current version, the change log, pending approvals, the request, and the step and cost budget used so far.
- **Actions:** tool calls, `ask_user`, and `finish`.
- **Observations:** tool results, validator verdicts and user decisions.
- **Stopping:** the run ends when the planner calls `finish` and no check is failing, or when the budget runs out (we plan 30 tool calls and a cost cap per task). In that case DataSage returns a partial report that lists unresolved issues instead of guessing.

We do not train the model with rewards. The scores in Section 6 are only used to evaluate and compare designs.

3.5 Memory and knowledge

Short-term memory is the session state. Instead of the full history of raw tool outputs, the model gets a compact summary of the state that is rebuilt at every step. Knowledge comes from two places: the check library, where each check is a small documented function the planner can choose, and the optional rules file, which the planner reads through `lookup_rule(column)`. As a Phase 4 stretch goal, an approved cleaning plan can be saved as a "recipe" and replayed on next month's file with the same columns. This is our candidate for the reusable capability the course asks us to ship back.

3.6 One agent, not several

We plan a single planner agent. The task is sequential and works on one shared table, so splitting it across several agents would add coordination cost without a clear benefit. The validator is plain code rather than a second agent, because row counts and totals should be checked exactly. If evaluation shows that the planner misses problems, we will test adding a reviewer model call that reads the profile and the change log and points out anything suspicious.

3.7 Models, frameworks and error handling

We will use Python 3.11+, pandas, openpyxl and pydantic, with a function-calling language model through an API. In Phase 2 we will compare one small, low-cost model with one larger model and choose based on accuracy, cost and tool-call reliability. We will write the agent loop ourselves (a few hundred lines) instead of using a large framework, so we fully control the state, the approval gate and the logging. The first interface is a command line; a simple Streamlit page is planned for the stakeholder demo. Failures are handled as follows:

- Invalid arguments or a tool exception: the error text goes back to the model (at most two retries per action).
- Failed validation: roll back to the previous version and tell the model. Sandbox code has a 30-second limit.
- API errors: retry with backoff, then stop with a partial report. If the same action fails twice, the issue is flagged and the planner must move on.

Section 4. Technical Challenges and Innovation

Much of DataSage is integration work: pandas, a model API and a command-line tool. Four parts go beyond that.

1. **Deciding between fix, flag and ask.** Too many questions make the tool annoying; too few make it unsafe. We enforce the risk tiers in code and always report two numbers together:

unsafe changes (target zero) and questions per file (target low). Improving one by hurting the other does not count.

2. **Showing that cleaning did not change the meaning of the data.** The validator compares each version cell by cell with the previous one, so any change that no logged action explains is caught. This also gives an exact count of unsupported changes, the failure mode named in the stakeholder's proposal.
3. **Keeping numbers honest.** Language models can write numbers that sound right but are wrong. In DataSage every number comes from a tool, and the final check rejects a summary that contains a number no tool produced.
4. **Tables larger than the context.** The model only sees a profile and a few sample rows, so those must reveal the problems. Choosing which rows to show, such as rows that failed a check, is a real design problem.

Other hard cases are dates like 03/04/2026 that can be read two ways, labels such as "Elec." that may or may not mean "Electronics", and negative values that may be refunds. Fuzzy label matching can merge labels that are really different, so we treat it as high risk.

What is new. No single piece is new on its own. What we find interesting is the combination: a model makes the judgment calls, while every change is versioned, checked, reversible and exported as plain pandas code. The user gets an answer plus an audit trail they can rerun without the agent.

Assumptions, risks and dependencies.

- Each file fits in memory and has one header row; sheets with merged cells or several tables are out of scope.
- We depend on a model API, so cost and rate limits matter; every task has a budget.
- Profiles and sample rows go to the API provider, so we use public or synthetic data unless the stakeholder agrees otherwise, with an option to hide sample values.
- We depend on the stakeholder for real files and output reviews; if he cannot share real data, we will use public messy datasets.

Section 5. Refined Baseline

B0: the stakeholder's baseline, kept unchanged. `run_baseline.py` loads a CSV or Excel file with pandas, runs fixed checks (missing values, exact duplicate rows, negative numbers, invalid values in columns whose names contain "date" or "time", and labels that differ only by case), runs one fixed analysis (revenue by category) and writes a Markdown report. It never changes the data. We copied it unchanged into our repository (baseline/, commit 4b1df80) and ran it with Python 3.13 and pandas 3.0.5: it reproduced the proposal's numbers exactly and its smoke tests passed. It needs Python 3.11 or newer, because pandas 3 does not support older versions.

Why we refine it. B0 does not clean, so its analysis inherits every problem it finds; its fixed checks miss problems outside its rules (three of four in our probe file); and it has one analysis, no approval step and no validation. Comparing our agent only against B0 would overstate the gain, since almost any cleaning beats it on accuracy. So we add two simpler baselines, each testing one idea (Table 3).

Table 3. Baselines.

Baseline	What it is	What the comparison shows
B0: stakeholder baseline (rule-based)	Fixed checks, no cleaning, one fixed analysis.	The gain over the stakeholder's current tool.

Baseline	What it is	What the comparison shows
B1: fixed cleaning script (rule-based, no model)	B0's checks plus fixes that are always applied: trim and lowercase labels, drop exact duplicates, turn invalid dates into missing values, drop rows with negative numbers. It gets each request already in structured form (group by, metric, aggregate, filter), because it cannot read plain English.	Whether the agent's per-file decisions beat a careful fixed script. B1 is given the structured request for free, so beating it is a fair test of the cleaning decisions.
B2: single model call	The same model as the agent, given the profile, sample rows and request in one prompt. It returns a cleaning plan and pandas code that we run once: no checks, no retries, no approval.	Whether the loop, tools and validation add value over "just ask the model".

Section 6. Dataset and Evaluation Plan

Data.

1. **Planted-error benchmark (main test set).** We take clean public tables with clear reuse licenses (for example from the UCI Machine Learning Repository or data.gov; each license is recorded) and a generator script plants errors of each in-scope type at known rates (1 to 5% of rows), recording every planted error (row, column, type, original value). For each table we write two or three questions and compute the answers on the clean table. Target: about 40 tables and 120 tasks, split into a development set and a held-out test set used only for final numbers.
2. **Real messy files.** Five to ten files with problems we did not plant, from the stakeholder and public sources, labeled separately by two team members who then settle disagreements. These test DataSage beyond our own generator.
3. **Fixtures.** The stakeholder's messy_sales files and our probe file, used in automated tests.

Privacy and licensing. We use only public or synthetic data and record each license in the repository. Any non-public file from the stakeholder stays out of the repository and is not sent to a model API without his permission.

Systems compared. B0, B1, B2 and DataSage, plus two ablations: DataSage without the validator, and DataSage with every risky fix approved automatically. The ablations show what each safety part adds.

Approvals during automated runs. No person answers approval questions in batch runs, so we use an approval oracle built from the planted-error records: it approves a risky fix only if the fix repairs a planted error. We also report a run where every risky fix is rejected, to check that DataSage stays correct, if less complete, when the user says no.

Table 4. Metrics and proposed targets (to be agreed with the stakeholder, Table 1 item 6).

Metric	How we measure it	Proposed target
Final task performance		
Analysis accuracy	Answer table matches the answer on the clean data (each value within 0.5%)	≥ 90% of test tasks
Detection precision and recall	Planted problems found, matched by row, column and type	Both ≥ 0.85
Correction accuracy	Fixed cells equal the clean value	≥ 0.80 of planted errors that should be fixed
Unsafe changes	Cells changed without a logged, approved reason, or valid cells damaged	0 unapproved destructive changes; < 1% of valid cells changed
Agent behavior		

Metric	How we measure it	Proposed target
Task completion	Answer and report produced within the budget	$\geq 95\%$
Tool-call success	Valid tool calls divided by all tool calls (from the trace log)	$\geq 95\%$
Recovery	Failures (tool errors, failed validations) followed by a successful next step	$\geq 80\%$
Questions per file	Approval prompts per task	Median ≤ 3
Latency	Wall-clock time per task	Median ≤ 60 s
Cost	Model API cost per task	Median $\leq \$0.10$

What counts as meaningful improvement. On the held-out test set, DataSage should beat B1 and B2 on analysis accuracy by at least 15 percentage points, with zero unapproved destructive changes, while staying within the latency and cost targets. All targets above are our proposal. [They become final once the stakeholder agrees; record any changes here.]

Human evaluation. After Phase 3 the stakeholder will rate DataSage's reports on 10 tasks for correctness, clarity and trust (1 to 5 each), and in Phase 4 he will run the acceptance test on files of his choice.

Plan for the rest of the semester.

- **Phase 2 (weeks 4 to 6):** profiler, check and fix tools, the agent loop, the error generator and the first 40 tasks; run B0, B1 and B2.
- **Phase 3 (weeks 7 to 10):** approval gate, validator, sandbox, automated tests, the evaluation harness and trace-log monitoring; second stakeholder meeting.
- **Phase 4 (weeks 11 to 13):** stakeholder feedback on real files, the reusable cleaning-recipe feature and the acceptance test; third stakeholder meeting.

Appendix A. Stakeholder Meeting Notes

[Date, attendees and length. Short bullet notes from the meeting: his answers to the questions below, decisions on Table 1, and any action items.]

Agenda we used for the meeting:

- Who would use DataSage, and on what kinds of files? Can you share two or three real messy files, or point us to public ones?
- Which problem types matter most to you, and which can wait?
- Which changes may DataSage make without asking, and which always need approval (Table 2)?
- What did you mean by retrieving external definitions? Would a rules file cover it (Table 1, item 3)?
- Which analysis requests should it handle first?
- Are about 60 seconds and \$0.10 per file acceptable?
- Do you agree with the metrics and targets in Table 4? What would make you trust the report?